

Synchronizing with the Scheduler: Dual-Loop Congestion Control for 5G Uplink on Commodity Devices

Qiang Wu*

University at Buffalo SUNY
Buffalo, NY, USA
qwu26@buffalo.edu

Yuxin Liu*

University at Buffalo SUNY
Buffalo, NY, USA
yuxinliu@buffalo.edu

Tianyang Zhang

University at Buffalo SUNY
Buffalo, NY, USA
tzhang52@buffalo.edu

Haoran Wan

Princeton University
Princeton, NJ, USA
haoran.w@princeton.edu

Kyle Jamieson

Princeton University
Princeton, NJ, USA
kylej@princeton.edu

Yaxiong Xie†

University at Buffalo SUNY
Buffalo, NY, USA
yaxiong@buffalo.edu

Abstract

Current end-to-end congestion-control feedback is too slow to track rapid wireless dynamics in cellular networks. We identify Grant-to-Buffer Ratio (GBR)—the ratio of base-station uplink grants to mobile-reported demand—as a millisecond-scale RAN signal of uplink resource scarcity. Measurements across AT&T, Verizon, and T-Mobile LTE/5G FDD/TDD deployments show that GBR tracks base-station uplink load and reveals congestion earlier than end-to-end feedback. Because GBR is derived from the mandatory BSR-grant exchange, it requires no base-station changes and captures scheduler decisions at their native timescale. We then design GBR-CC, a dual-loop controller that updates the sender rate on each GBR sample, using GBR for fast adaptation and end-to-end delay trends as a conservative fallback. This design lets the sender react before queues inflate while still handling non-radio bottlenecks through the outer loop. GBR-CC runs on commodity mobile devices without extra hardware or external tools. Experiments on commercial cellular networks show that GBR-CC improves average throughput over GCC by 50%, while reducing median playout latency by 32–53% and freeze rate by 60%; compared with BBR, it improves average throughput by 5% and halves median RTT.

CCS Concepts

• **Networks** → **Mobile networks**; **Network protocol design**.

Keywords

Radio Access Network, Congestion Control, 5G, LTE

ACM Reference Format:

Qiang Wu, Yuxin Liu, Tianyang Zhang, Haoran Wan, Kyle Jamieson, and Yaxiong Xie. 2026. Synchronizing with the Scheduler: Dual-Loop Congestion Control for 5G Uplink on Commodity Devices. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3789240.3829199>

*Co-primary authors.

†Yaxiong Xie is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM '26*, August 17–21, 2026, Denver, CO, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/26/08

<https://doi.org/10.1145/3789240.3829199>

1 Introduction

Cellular networks are experiencing a surge in uplink-intensive, real-time applications.¹ From high-definition video conferencing [10, 20, 28] and mobile live streaming (e.g., Twitch [40], TikTok [39]) to teleoperated robotics [7, 18], these services demand consistent high throughput and low latency from the device to the cloud. However, the cellular uplink remains a critical bottleneck. Unlike the downlink, where base stations utilize high transmission power, large antenna arrays, and complex signal processing, the uplink is strictly limited by the user equipment's (UE) restricted size, battery power, and computational capacity. As a result, the uplink is inherently vulnerable: even slight variations in signal strength or multi-user contention can cause immediate capacity drops [19, 24, 49].

In such a constrained environment, the congestion control (CC) algorithm is the primary line of defense. However, achieving low-latency, high-throughput uplink transmission remains an elusive goal for existing transport protocols. The root cause is their inability to track the rapidly changing cell load due to a fundamental timescale mismatch between the transport layer and the Radio Access Network (RAN). While the RAN scheduler reallocates resources on a sub-millisecond timescale [5, 30, 48], traditional transport protocols [8, 10, 11, 13, 16, 22, 43, 44, 50, 54] rely on end-to-end feedback loops that are often orders of magnitude slower. This inherent lag forces existing algorithms to drive blind: by the time an end-to-end signal reports congestion, the radio state has already shifted. As a result, senders inevitably oscillate between overdriving the link (causing bufferbloat) and underutilizing available capacity [24, 25, 37, 49, 55].

To bridge this gap, a significant body of research relies on explicit feedback or in-network assistance from the cellular infrastructure. Proposals like L4S [12, 42] and cellular-specific ECN [15, 21, 26, 35] utilize explicit congestion notification to signal saturation. Octopus [14] leverages in-network assistance to drop packets or adapt content proactively. While these methods offer high accuracy by directly exposing the base station's state, they face prohibitive deployment barriers: they require modifications to proprietary, closed-source base station hardware.

Alternatively, researchers have explored client-side cross-layer approaches to infer channel quality. Approaches like NG-Scope [47], NR-Scope [41], and piStream [45] use physical layer sniffing but

¹GBR-CC is available at <https://github.com/NEMO-LAB-repo/GBR-CC>.

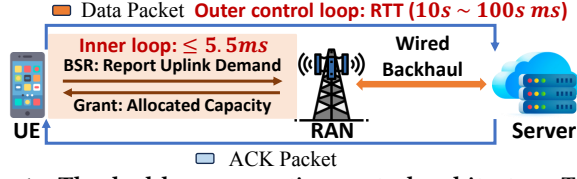


Fig. 1 – The dual-loop congestion control architecture. Traditional RTT-scale outer-loop control is mismatched to rapid wireless dynamics and reacts only after queues build up. We introduce a millisecond-scale inner loop that leverages the RAN-internal BSR-Grant process to infer resource scarcity and adapt the sending rate at the timescale of base-station scheduling, while retaining the outer loop for end-to-end fallback.

require expensive external hardware, e.g., software-defined radios like USRP, making them impractical for ubiquitous deployment. Conversely, tools like MobileInsight [30, 31] operate on commodity smartphones by decoding control-plane signaling; however, monitoring a single device provides no visibility into the demands of other users. Since a device can only observe its own state, it cannot infer total cell load or reliably distinguish between self-inflicted congestion and multi-user contention.

In this paper, we ask: *Is it possible to infer the precise load of the base station directly from a commodity mobile device, in real-time, without infrastructure support?*

We answer affirmatively by identifying a fast, closed-loop control mechanism native to the cellular architecture: the *buffer status report* (BSR) to *uplink grant* loop [1]. In cellular protocols, both LTE and 5G, a UE cannot transmit without permission; it must first report its queue size via BSR to the base station, which then allocates resources based on availability and notifies the UE via uplink grants. We observe that the ratio between what the device asks for and what the network grants serves as a high-fidelity, instantaneous proxy for cell load. Intuitively, when a cell is lightly loaded, the base station can fully satisfy the device’s uplink demand; however, as contention rises and multiple users compete for finite radio resources, the scheduler must ration grants, causing the allocated capacity to drop below the reported demand.

This mechanism offers decisive advantages over traditional feedback signals. First, it enables a millisecond-scale control loop capable of reacting to rapid wireless dynamics, by providing updates at a high frequency (approximately every 5 ms). Second, unlike data packets which get trapped in standing queues, BSR and Grant messages travel on dedicated control channels, rendering them immune to the buffering delays (RTT inflation) that make end-to-end signals stale. Finally, this approach leverages standard collaboration from the base station without requiring any modifications to cellular infrastructure, as the telemetry is accessible on commodity UE via standard diagnostic interfaces.

We formalize this insight as the Grant-to-BSR Ratio (GBR), defined as the ratio of the allocated uplink Grant size to the queue depth reported via BSR. To verify that GBR is a reliable indicator of instantaneous cell load, we conducted an extensive half-year measurement campaign across three major US carriers, i.e., AT&T, Verizon, T-Mobile, covering diverse LTE and 5G (FDD and TDD) deployments. By systematically varying background traffic to modulate the actual cell load, we confirmed a strong negative correlation

between the GBR and the cell load. Our measurements show that while the cell is uncongested, the GBR holds at ≥ 1 , i.e., granted resource is larger than UE requested; crucially, as the cell saturates and the scheduler forces throttling, the GBR drops proportionally below one, confirming that the GBR is a high-fidelity proxy for base station saturation.

Guided by this insight, we design and implement GBR-CC, a congestion control algorithm with a dual-loop architecture that decouples adaptation to wireless capacity from end-to-end dynamics, as shown in Figure 1. The core innovation is the millisecond-scale inner loop, which acts as a fast-response traffic shaper. By monitoring the GBR in real-time, this loop continuously estimates the available physical capacity and adjusts the packet pacing rate accordingly before queues accumulate. This transforms congestion control from a reactive process, i.e., waiting for standing queues to trigger end-to-end signals, into a proactive one that synchronizes the sender with the base station’s scheduler. Simultaneously, a traditional outer loop monitors end-to-end delay trends to handle non-radio bottlenecks e.g., shared backhaul congestion.

To demonstrate the applicability of GBR-CC to the real-time interactive applications, we merge it into WebRTC stack. Directly applying the aggressive capacity tracking of GBR-CC to video traffic is suboptimal; the resulting rapid bitrate fluctuations destabilize the video encoder, causing the receiver’s jitter buffer to inflate and unintentionally degrade playout latency. Therefore, we introduce a QoE-aware tuning that tunes the controller’s rate adaptation aggressiveness. We desensitize the controller to minor channel variations to avoid unnecessary rate oscillations. This modification prioritizes rate stability over perfect bandwidth matching during minor fluctuations, preventing jitter buffer bloat while retaining the ability to react instantly to congestion.

We implement GBR-CC on commodity Android smartphones. Extensive experiments across commercial LTE and 5G networks demonstrate that GBR-CC significantly outperforms state-of-the-art baselines: in bulk transport scenarios, it reduces median RTT by up to $4\times$ compared to loss-based schemes while delivering $2.5\times$ the throughput of delay-based alternatives. When integrated into WebRTC for real-time video, GBR-CC reduces median end-to-end playout latency by 32–53% and cuts video freeze rates by approximately 60%, effectively eliminating the stalls that plague interactive applications in dynamic cellular environments.

Ethics. This work does not raise any ethical issues.

Contributions. Our contributions are as follows:

- **GBR Congestion Signal.** We identify the GBR, a novel millisecond-scale proxy for cell load accessible on commodity smartphones without infrastructure modifications.
- **Measurement-Driven Validation.** Extensive campaigns across AT&T, Verizon, and T-Mobile confirm GBR strongly correlates with base station load, detecting congestion orders of magnitude faster than end-to-end feedback.
- **Dual-Loop Congestion Control.** We design GBR-CC, which integrates a millisecond-scale inner loop for proactive rate adaptation and an end-to-end outer loop to handle non-radio bottlenecks. We adapt GBR-CC to WebRTC through QoE-aware tuning, enabling scheduler-level RAN signals to guide real-time video without destabilizing the encoder and jitter buffer.

2 Background and Motivation

This section explains the limitations of existing cellular congestion-control approaches, then introduces GBR as a practical scheduler-level signal for building a dual-loop congestion controller.

2.1 Congestion Control in Cellular Network.

Traditional congestion control algorithms, whether loss-based (e.g., CUBIC [22]), delay-based (e.g., GCC [10], Verus [54], Sprout [44]), or model-based (e.g., BBR [13]), fundamentally struggle in cellular environments due to two fundamental problems with end-to-end feedback. First, these signals are lagged: structurally, they are too slow to track rapid wireless dynamics. While the RAN scheduler reallocates resources at sub-millisecond timescales, transport feedback is governed by the Round-Trip Time (RTT), which is frequently inflated by deep buffers to hundreds of milliseconds, forcing protocols to react to network states that have already shifted. Second, these signals are inaccurate: they lack visibility into RAN-internal information. Because endpoints treat the network as a black box, they are blind to the base station's actual scheduling decisions and resource availability. This forces algorithms to guess the congestion state based on noisy symptoms (delay or loss) rather than the root cause, leading to persistent rate mismatches.

2.1.1 Network-Side Signaling

Ideally, the base station—which possesses complete knowledge of network dynamics—would explicitly notify senders of congestion. Proposals such as ABC [21] and Octopus [14] suggest modifying the base station to mark packets with capacity information. While these methods help to accurately estimate network capacity, they cannot shorten the control loop, meaning the estimation still risks becoming outdated by the time it reaches the sender. Zhuge [35] attempts to shorten the feedback loop by having the base station predict queuing delays and generate feedback immediately, thereby eliminating the latency associated with packet queuing and air interface transmission. The architecture of Zhuge limits itself to downlink traffic. More importantly, these solutions face prohibitive deployment barriers. They require modifications to closed, proprietary RAN infrastructure and rely on changing 3GPP standards, which is impractical for immediate deployment.

2.1.2 UE-Side Cross-Layer Solution

To address this, recent research has pivoted toward cross-layer approaches that expose RAN internals at the UE side. We classify these into two paradigms: *hardware-based global sniffing* and *client-side local telemetry*.

The Gold Standard: Hardware-Based Global Visibility. The theoretical ideal for congestion control is perfect visibility into the base station's load. Tools like NG-Scope [47], NR-Scope [41], and piStream [45] achieve this by monitoring the physical downlink control channel or physical spectrum energy using Software-Defined Radios (SDRs). PBE-CC [48] builds a congestion control algorithm atop NG-Scope [47] and demonstrate superior performance. While scientifically robust, this approach requires expensive and bulky external hardware (USRPs), making it deployment-prohibitive. Furthermore, decoding the control channels of all users raises significant privacy concerns in commercial networks.

UE-Side Local Telemetry. Instead of external hardware, software-based tools like MobileInsight [30], QCSuper [38], XCAL [6], extract

internal metrics directly from the mobile device's modem. Various algorithms leverage these metrics to guess network conditions: CLAW [46] correlates signal quality (RSRQ) with base station load; PERCEIVE [29] uses machine learning on resource allocation patterns; H2MO [33] detects handovers to predict capacity drops; and Prism5G [51] tracks carrier aggregation to anticipate bandwidth jumps, e.g., when a secondary carrier is activated, capacity might jump from 50 Mbps to 500 Mbps instantly.

However, these methods face two critical limitations. First, and most importantly, they lack a holistic view. Because a base station serves many users simultaneously, the metrics observed by a single device, such as its own allocated bandwidth or signal interference, cannot capture the aggregate contention in the cell. Second, existing extraction tools generally cannot expose this data in real-time, often introducing seconds of delay that make the signals useless for real-time active congestion control, so that most of the algorithms are tested using trace-driven emulation.

2.2 New Vantage Point: The BSR-Grant Loop

The cellular network architecture inherently contains a fast, closed-loop control mechanism that can be leveraged to infer base station load without infrastructure modifications.

2.2.1 BSR-to-Grant Loop

Cellular networks are centralized architectures where a mobile device cannot transmit uplink data without explicit permission. The cycle begins when data enters the UE's uplink buffer, triggering the UE to send a BSR to the base station indicating the volume of queued data. In response, the base station scheduler evaluates this request against available resources, issuing an uplink grant that specifies exactly how many bytes the UE is permitted to transmit.

Crucially, this interaction reveals the instantaneous load of the cell. The mobile device always requests the resources necessary to clear its buffer. Whether the base station grants this full amount depends heavily on its current load. When the base station is not saturated and possesses sufficient bandwidth, the granted resources match the reported demand. Conversely, when the cell is congested, either by the device's own traffic or aggregate demand from multiple users, the granted resources fall short of the request. This discrepancy serves as a direct indicator of base station saturation.

2.2.2 New Signal of Congestion: Grant-to-BSR Ratio (GBR)

The relationship between what a UE asks for (BSR) and what it receives (Grant) provides a direct signal of base station saturation. We define the Grant-to-BSR Ratio (GBR) η as:

$$\eta = \frac{R_{grant}}{R_{bsr}} = \frac{\text{Cell Granted Resource}}{\text{BSR Requested Resource}} \quad (1)$$

where the R_{grant} is the resource granted by the cell and the R_{bsr} is the resource requested by the mobile device via BSR.

Signal Interpretation and Cell Load Correlation. The GBR acts as a proxy for the base station's scheduler state. When $\eta \geq 1$, the base station is uncongested, possessing sufficient bandwidth to satisfy the UE's demand immediately. Interestingly, η can slightly exceed 1 in this state because the base station may allocate extra padding to prevent fragmentation or to accommodate traffic arriving between the BSR and the grant. Conversely, when $\eta < 1$, the base station is saturated. In this regime, the scheduler cannot

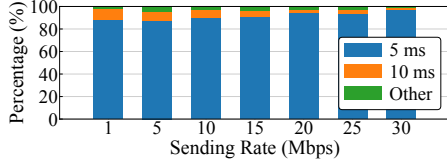


Fig. 2 – Distribution of interval between two BSR reports across three operators in US, spanning LTE and 5G networks with both FDD and TDD cells.

satisfy the aggregate demand, forcing it to clip the UE’s request; the lower the ratio, the more severe the congestion. This monotonic relationship transforms the ratio from a simple accounting metric into a high-fidelity congestion signal.

2.2.3 Advantages of the GBR Signal

The GBR offers five distinctive advantages that overcome the limitations of prior approaches.

Shortest Control Loop. The BSR-to-Grant RTT represents the tightest feedback loop in the cellular stack. In FDD, the base station can grant resources in the very next TTI. In TDD, despite frame structure constraints requiring a wait for a downlink slot, our empirical results confirm the delay is strictly bounded below 5.5 ms, orders of magnitude faster than transport-layer RTTs.

Frequent Signal Updates. To maintain uplink transmission, the mobile device must periodically report its buffer status. This periodicity is configured by the network through RRC signaling: the RRC Reconfiguration message carries the BSR-Config, which specifies the periodicBSR-Timer parameter [2, 4]. We obtain this value in real time by parsing the RRC Reconfiguration message. Appendix D describes the complete parsing workflow for computing GBR, including how uplink grants are extracted and aligned with BSR reports. In our traces, the base station sets this timer to 5 ms across three major U.S. operators. Since every BSR generates a corresponding grant evaluation, the GBR signal updates at this exact frequency, allowing the controller to track millisecond-scale wireless dynamics without relying on delayed end-to-end transport feedback.

Immunity to Buffer Bloat and Inflated RTT. BSRs and Grants are control-plane messages prioritized over user data. Even when the data plane is heavily congested and buffering (inflating RTT), the BSR-Grant loop continues to operate on its millisecond schedule, providing a lag-free signal of congestion. Consequently, the feedback loop continues to operate reliably on its predefined schedule even when the cell is saturated and data buffers are overflowing. Figure 2 validates this behavior across multiple operators. We increased the sending rate up to 30 Mbps, a load that significantly exceeds the available uplink capacity in all our tests. Despite this heavy saturation, the measured BSR interval remains tightly clustered around its 5 ms schedule, confirming that the BSR reporting interval operates independently of the cell load.

Availability at Commodity Mobile Device. The BSR and Grant messages required to compute GBR are accessible on standard commercial hardware through vendor diagnostic interfaces available on Qualcomm and MediaTek chipsets. Existing tools, e.g., Qualcomm DIAG [6], MobileInsight [30], and QCSuper [38], can access these interfaces and expose low-level modem messages. We have developed a tool, CellNinja, to parse these messages and expose the derived GBR in real time on modern 5G smartphones.

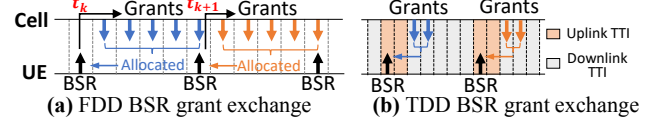


Fig. 3 – Temporal grant distribution following a BSR. (a) FDD allows allocation in any TTI. (b) TDD restricts allocation to specific slots defined by the frame structure.

Implicit Network Assistance. The GBR signal effectively provides network-assisted congestion control, leveraging the cell’s precise knowledge of cell load, without requiring any modifications to the cell itself or deployed base-station software. It exploits a standard, mandatory procedure defined in 3GPP specifications [1–4], making it immediately deployable on existing networks.

2.3 New Architecture: Dual Loop Control

Building on the discovery of the BSR-Grant Ratio, we propose a novel Dual-Loop Congestion Control architecture for cellular uplink traffic. As illustrated in Figure 1, this architecture integrates two distinct feedback mechanisms: a conventional outer loop governed by data packets and ACKs, and a fast inner loop driven by BSR and Grant messages.

Outer Loop. This loop, employed by traditional end-to-end algorithms, detects capacity across the entire network path. It remains essential for identifying bottlenecks that occur outside the RAN, such as in the wired backhaul.

Inner Loop. This loop explicitly targets the wireless connection between the UE and the base station. By monitoring the GBR, it provides immediate visibility into RAN-specific dynamics that end-to-end signals often miss or delay.

By synthesizing these two layers of information, our architecture achieves the best of both worlds: it rapidly adapts to fast-varying cellular dynamics via the inner loop, while retaining the ability to detect and react to non-radio bottlenecks situated in the wired backhaul via the outer loop.

3 GBR: The Formulation, Empirical Validation, and Benchmarking

In this section, we first formalize the GBR as a real-time congestion signal derived from the cellular inner control loop, then validate its correlation with base station load across commercial LTE and 5G networks. Finally, we leverage the GBR to benchmark existing congestion controllers.

3.1 GBR Formulation and Extraction

Grant Distribution and Aggregation. To rigorously formulate the GBR, we must account for the temporal fragmentation of resource allocation. Since each uplink grant permits transmission only within one specific TTI, the base station may distribute the total allocation across multiple discrete grants over several TTIs if a single TTI lacks sufficient resources due to spectral constraints or multi-user contention. This distribution varies by duplexing mode: FDD allows the scheduler to allocate resources in any TTI, potentially resulting in contiguous blocks of grants, as shown in Figure 3a; whereas TDD is constrained by its frame structure. Because the number of uplink TTIs is limited in TDD, the maximum

number of grants issued within a reporting period is accordingly restricted, as illustrated in Figure 3b.

Attributing these grants to the correct BSR is critical for accurate calculation of GBR. While the 3GPP protocol does not explicitly tag a grant with a *BSR ID*, the cell issues grants in response to receiving a new BSR and generally attempts to clear the reported buffer before the next periodic report arrives. We therefore use temporal boundaries to map grants to requests: all grants received between the transmission of a BSR at time t_k and the subsequent BSR at time t_{k+1} are attributed to the demand reported at t_k , as shown in Figure 3.

Formulating the GBR Metric. The Grant-to-BSR Ratio (η) is calculated as defined in Eq. 1. We calculate the specific components for a single BSR cycle as follows. As defined in Section 2, R_{bsr} represents the queue depth reported by the UE in a specific BSR message [1]. To determine the total resources allocated in response to this request, we aggregate all uplink grants received between the current BSR η_k and the subsequent BSR η_{k+1} . The total granted resource, R_{grant} , is defined as: $R_{grant} = \sum_{i=1}^N R_i^g$, where R_i^g denotes the allocated resource (in bytes) of the i -th grant, and N represents the total number of grants issued between η_k and η_{k+1} .

Real-Time Telemetry on Commercial Devices. Calculating the GBR requires real-time access to control plane messages. To obtain this data, we leverage CellNinja [32], a diagnostic tool capable of capturing and parsing BSR and grants from commodity smartphones; this currently requires root access on the smartphone. CellNinja provides submillisecond telemetry, enabling immediate GBR calculation. In our measurements, this parsing-and-computation path remains consistently below 1 ms. CellNinja's runtime overhead has been characterized in prior work [32]: it incurs only modest CPU overhead, around 2–3%, and slightly increases device power consumption across LTE and 5G SA networks.

3.2 Empirical Validation of GBR

To rigorously validate the GBR as an indicator of cellular resource scarcity, we conducted empirical measurements to correlate it with ground-truth base station load.

Methodology for Ground-Truth Load Estimation. As established in §2.1.2, the most accurate metric for cell load is the aggregate physical resource block (PRB) utilization across all served UEs. We utilized open-source telemetry tools, NG-Scope [47] for LTE and NR-Scope [41] for 5G NR, to decode the control messages sent by the base station to all active users. We define the cell load (L) as $L = \frac{P_a}{P_c}$, where P_a represents the total PRBs allocated by the cell to all UEs during each measurement window, and P_c is the total available PRB based on the cell's bandwidth.

Experimental Settings. We implemented CellNinja on a commodity Android smartphone to record real-time GBR samples while simultaneously performing uplink data transfers to an AWS server. To capture a diverse range of network states, we swept the application data rate from 80 kbps to 30 Mbps (exceeding the typical uplink capacity). Each experimental run lasted eight minutes, and we conducted measurements during both peak and off-peak hours across three nationwide operators: AT&T, Verizon, and T-Mobile. Our measurement campaign spanned from August 2025 to January

2026. In total, we collected over five hours of high-granularity data per operator and per radio configuration.

3.2.1 Visualizing the BSR and Uplink Grant Interaction

To illustrate the temporal dynamics of the GBR, Figure 4 plots the real-time requested demand (R_{bsr}) from UE and granted resources (R_{grant}) from the cell, across three representative 10-second traces with different uplink sending rates. We further validate the same BSR/Grant dynamics on a MediaTek-based device in Appendix A.

Uncongested Regime (Figure 4a). When the UE transmits at a low rate of 250 Kbps, the base station frequently over-allocates resources. Here, R_{grant} consistently exceeds R_{bsr} , resulting in an average GBR of 1.549.

Near-Capacity Regime (Figure 4b). At moderate rate of 12 Mbps, the scheduler tightly tracks the UE's demand. The granted resources closely match the reported buffer status, keeping the ratio near one (average GBR = 0.961).

Saturated Regime (Figure 4c). When the sending rate exceeds available capacity (20 Mbps), the scheduler is forced to clip the UE's requests to accommodate physical layer limits or other users. R_{grant} falls significantly below R_{bsr} over sustained intervals, and the GBR collapses to nearly zero (average 0.099), providing an unmistakable signal of RAN-internal congestion.

3.2.2 Characterizing the GBR-Load Relationship

We aggregate GBR and cell load values into discrete time windows of 50 ms and plot the distribution of GBR versus ground-truth cell load for 5G and LTE in Figures 5.

FDD Networks (LTE & 5G). Across T-Mobile, AT&T, and Verizon FDD deployments, we observed a consistent and sharply monotonic dependence. When the GBR $\eta < 1$, the cell load L remains at a near-saturation level. Conversely, when $\eta > 1$, the cell load is minimal, reflecting an idle or underutilized state. A rapid phase transition occurs at $\eta \approx 1$, where the load drops precipitously.

TDD Networks (T-Mobile 5G). This configuration (Figure 4b) exhibits a distinct linear trend. Rather than a binary transition, the GBR decreases linearly as the cell load increases. This linear characteristic provides an opportunity to more accurately estimate the specific value of cell load L based on the observed ratio. We attribute this difference in behavior to the specific scheduling algorithms employed in TDD versus FDD deployments.

These results demonstrate that the GBR is a robust and high-fidelity indicator of cell load across various duplexing modes and network generations. While the exact numerical mapping varies by scheduler type, the GBR consistently signals the congestion level of the base station.

3.3 Benchmarking Controllers with GBR

We utilize the GBR as a high-fidelity ground-truth indicator, To evaluate the efficacy of current CC in cellular uplink. By correlating GBR with the behavior of BBR [13] and GCC, the default CC for WebRTC, we expose the fundamental mechanisms that lead to their suboptimal performance in the presence of rapid RAN dynamics.

3.3.1 Model-Based Inaccuracy: The BBR Overshoot

BBR aims to operate at the "Kelvin point" by simultaneously achieving maximum throughput and minimum RTT. It estimates bottleneck bandwidth from the maximum delivery rate observed over

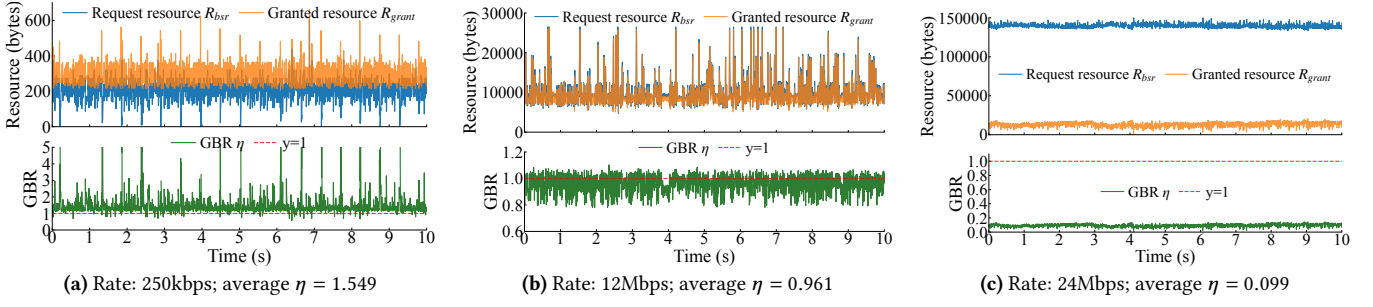


Fig. 4 – Temporal dynamics of requested resource R_{bsr} and granted resources R_{grant} across three load regimes. (a) Uncongested: grants exceed demand via scheduler padding ($\eta > 1$). (b) Matched: grants tightly track demand at the link capacity ($\eta \approx 1$). (c) Saturated: grants are clipped below requested demand due to cell congestion ($\eta \ll 1$).

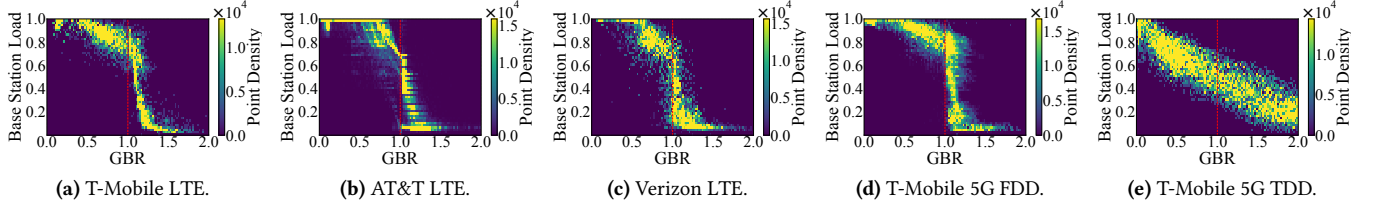


Fig. 5 – Relationship between base station load and GBR for T-Mobile, AT&T and Verizon, spanning LTE and 5G networks.

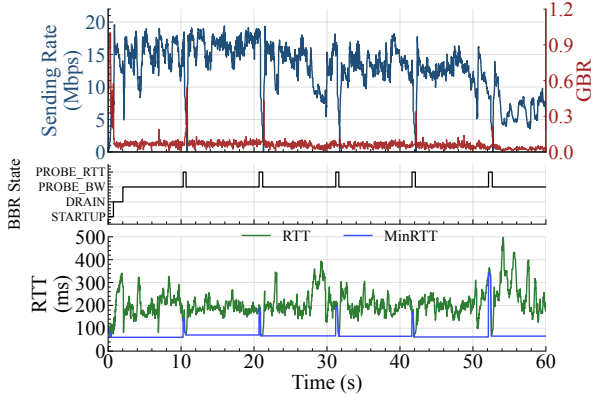


Fig. 6 – BBR on a cellular uplink. Top: sending rate (blue) and GBR (red). Middle: BBR state machine transitions. Bottom: RTT over time. The GBR remains near zero except during PROBE_RTT phases, indicating persistent over-sending; RTT stays elevated at 200–350 ms throughout.

a sliding window. However, on cellular uplinks, this estimate frequently becomes stale due to the millisecond-scale variability of the RAN scheduler.

As illustrated in Figure 6, BBR’s behavior reveals a persistent mismatch between the sender and the RAN across its internal state machine. After the initial STARTUP phase (exponentially probing for bandwidth) and the DRAIN phase (clearing the resulting startup queue), BBR enters its steady-state PROBE_BW phase. In this regime, it paces at its estimated capacity. However, our measurements show that the GBR (η) remains anchored near zero throughout these phases, signaling that BBR’s estimated capacity is consistently higher than what the base station is actually granting.

This disparity leads to massive queue accumulation at the base station, inflating RTT to 200–350 ms despite high throughput. The transient spikes in GBR during the PROBE_RTT phase, where BBR briefly drains the path by capping its congestion window to a small minimum flight size to refresh its minimum-RTT estimate, confirm that the congestion is self-inflicted rather than a transient radio fade: as soon as BBR resumes its normal pacing, the GBR collapses again, indicating immediate RAN saturation.

3.3.2 Delay-Based Blindness: GCC’s Over- and Underuse

GCC utilizes delay gradients to detect congestion: a sustained positive slope in one-way delay triggers an *overuse* signal. However, this approach remains blind to the internal scheduling dynamics of the RAN, and the GBR reveals two systematic failure modes in this logic for uplink traffic.

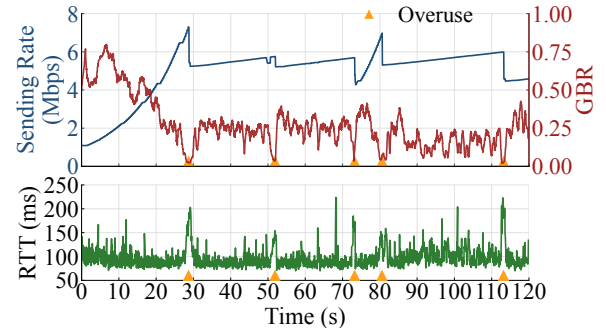


Fig. 7 – GCC’s disconnect from RAN dynamics: blind overshoot and inefficient recovery. GCC ignores the collapsing GBR (red), driving the rate (blue) into saturation and spiking RTT before reacting. It then underutilizes available bandwidth (high GBR) due to a slow, conservative additive recovery.

Unforeseen Saturation and Delayed Feedback (Overuse). As shown in Figure 7, the failure of GCC to avoid congestion before it happens stems from its lack of visibility into the RAN’s scheduling state, leading to a multi-stage breakdown. First, as GCC probes for bandwidth, the cell load increases and the GBR drops accordingly; this is a clear, early indicator of resource scarcity. Second, because GCC cannot observe this drop, it remains oblivious to the mounting load and continues its multiplicative rate increase. Third, GCC only becomes aware of the congestion once the cell is fully saturated and a significant delay spike occurs (Figure 7 lower). Fourth, this delay signal must then traverse the long end-to-end RTT loop before reaching the sender. Consequently, GCC is incapable of taking preemptive action to stabilize the queue, resulting in a severe overshoot that persists for at least one full RTT, significantly degrading real-time interactivity.

Inefficient Recovery (Underuse). When RAN resources become available, the GBR rises accordingly (Figure 7 upper), indicating that the base station can support higher demand. However, because GCC is designed to be conservative following a delay spike, it switches to a slow *additive increase*. This recovery phase can persist for *tens of seconds* before the sender reaches the new available capacity, leaving the RAN significantly underutilized and causing a perceptible drop in video quality during the recovery period.

3.3.3 Motivating GBR-Driven Proactive Adaptation

The systematic failures of BBR and GCC highlight a fundamental need for a controller that operates at the same timescale as the RAN’s internal scheduling. By incorporating the GBR as a fast, scheduler-facing signal, we can transform congestion control from a reactive process into a proactive one. Unlike end-to-end delay, which only signals that a queue has already formed, the GBR provides a real-time view of RAN resource scarcity, allowing a controller to detect the onset of saturation the moment the base station begins clipping grants relative to demand.

4 GBR-Driven Controller Design

We present GBR-CC, a dual-loop architecture that bridges the timescale gap between transport feedback and cellular dynamics. By integrating a fast, millisecond-scale inner loop driven by the GBR with a conventional outer loop, GBR-CC proactively adapts to RAN scheduling. We first detail the core algorithm (§4.1), designed to maximize throughput and minimize network latency. We then adapt this design for WebRTC (§4.2), employing a QoE-aware tuning that maximizes the QoE of real-time video conferencing.

4.1 Design of GBR-CC

4.1.1 Design Goal and Target Operating Point

The primary objective of GBR-CC is to maximize uplink utilization without inducing deep buffer saturation. Ideally, one might target a GBR of $\eta \approx 1$, where every request is immediately granted. However, our empirical analysis (Figure 5) reveals that for both FDD and TDD networks in practice, $\eta \approx 1$ correlates with underutilization, as the cell typically reaches full load only when $\eta < 1$, due to the scheduler’s algorithm and allocation policy.

Furthermore, while the BSR-Grant control loop is fast (≈ 5 ms), the delay is not zero. Targeting $\eta = 1$ leaves no margin for this lag; if channel capacity spikes during the update interval, a UE

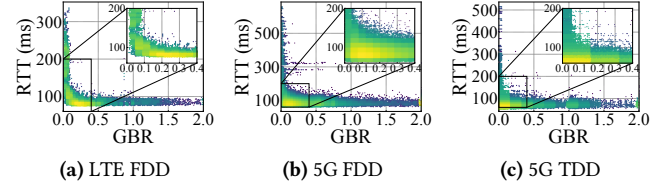


Fig. 8 – Empirical relationship between the GBR and RTT across (a) LTE FDD, (b) 5G FDD, and (c) 5G TDD networks.

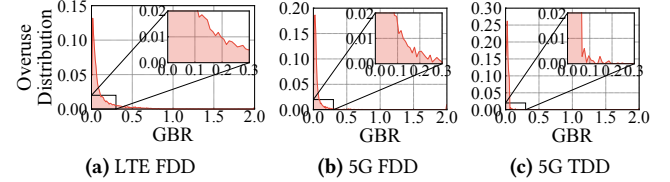


Fig. 9 – Distribution of WebRTC GCC overuse events relative to the GBR for (a) LTE FDD, (b) 5G FDD, and (c) 5G TDD.

with an empty buffer cannot exploit the extra slots. To ensure high utilization, we must therefore target an operating point $\eta^* < 1$. This maintains a persistent, shallow backlog, effectively greasing the pipe to bridge the control loop latency, creating a state of controlled saturation where demand slightly exceeds service. Since commercial schedulers are proprietary black boxes preventing theoretical derivation, we identify the target operating GBR η^* empirically by analyzing the trade-off between GBR and congestion metrics to find the optimal *knee of the curve*.

Empirical Derivation of η^* . To empirically determine the optimal target η^* , we analyze the relationship between GBR and network congestion using data collected from two distinct sources: our measurement campaign in §3.2 and an extensive WebRTC measurement campaign spanning over 30 hours per operator per radio configuration. We utilize two congestion indicators: RTT that serves as a direct proxy for queuing delay accumulated when the sender exceeds the service rate, and the *overuse event* detected by WebRTC’s GCC algorithm, which is specific to the WebRTC experiments.

Figure 8 plots the RTT against the GBR, and Figure 9 plots the detected overuse against the GBR. We observe a consistent phase transition characterized by a *cliff* below $\eta < 0.2$, where RTT variance explodes, mean RTT rises sharply, and GCC overuse events are concentrated due to deep queue buildup at the base station from the sender consistently exceeding allocated grants. As the ratio rises into the *stable region* ($\eta \geq 0.2$), queue buildup becomes less severe and RTT stabilizes. Based on these observations, we select $\eta^* = 0.2$ as the target congestion threshold. This operating point represents the empirical *knee of the curve*, aggressive enough to ensure the cell is fully utilized, but sufficiently buffered from the cliff where latency degrades uncontrollably. Appendix B further evaluates the sensitivity of this target.

4.1.2 Congestion Control Algorithm.

GBR-CC functions as a rate-based controller. It updates the sending rate S upon the arrival of every new GBR sample η , approximately every 5 ms, as shown in Figure 2.

Rate Adaptation Law. The controller employs a multiplicative update rule to adjust the sending rate from S_k to S_{k+1} after receiving

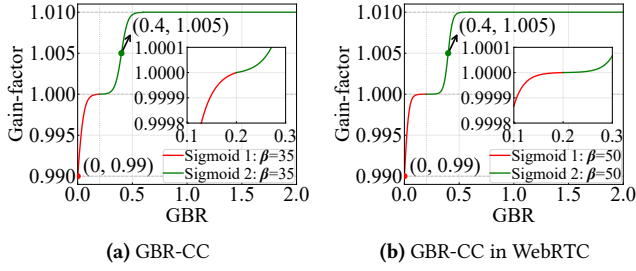


Fig. 10 – The piecewise sigmoid functions used to map GBR values η to rate adaptation gains $\gamma(\eta)$ for (a) GBR-CC and (b) GBR-CC in WebRTC.

$(k + 1)^{th}$ GBR η_{k+1} using a gain factor $\gamma(\eta)$:

$$S_{k+1} = \gamma(\eta_{k+1}) \cdot S_k. \quad (2)$$

If $\eta_{k+1} > \eta^*$, the gain factor $\gamma > 1$, increasing the rate to exploit available capacity, and vice versa.

Sigmoid Gain Mapping. Determining the magnitude of γ is critical. Our design objective is two-fold. First, we aim to create a *neutral zone* around the target η^* . Within this zone, rate adjustments should be minimal to prevent the ping-pong oscillations that arise from large step sizes in high-frequency control loops. Second, outside this zone, when the GBR η deviates significantly from the target η^* , we require aggressive adaptation. In these regimes, the controller must increase or decrease the rate rapidly to drive the system back toward the optimal operating point. Naive approaches fall short of this dual requirement. Fixed step sizes lack flexibility, causing ping-pong oscillations near the target, while linear mappings force a trade-off where a slope shallow enough for stability is too slow to react to deep congestion.

To address this, we employ a piecewise function composed of two shifted sigmoids. By offsetting the centers of these sigmoids away from the target η^* , we position their saturated tails, the regions where the curve flattens out, around the target, as shown in Figure 10a. This creates the desired neutral zone: when η is close to η^* , the function operates in these flat tails, resulting in minimal rate adjustments ($\gamma \approx 1$) that suppress oscillation. As the GBR deviates further from the target, the function transitions into the sigmoid’s steep central region, providing the necessary aggressiveness to correct the rate quickly. However, we must prevent this aggressiveness from becoming unbounded and thus causing dangerous rate increasing or decreasing spikes.

We strictly bound the rate adjustment per step to $\pm 1\%$, i.e., $\gamma \in [0.99, 1.01]$ by leveraging the sigmoid function’s natural saturation. As η approaches 0, the function operates in the steep transition region of the first sigmoid. This allows the gain to drop deeply to 0.99, ensuring aggressive rate reduction to quickly resolve saturation. As η becomes very large, the second sigmoid approaches its upper asymptote (Figure 10a). We scale this to a maximum gain of 1.01, ensuring the rate increases by at most 1% per step. This constraint is vital because the control loop runs at 200 Hz (5 ms intervals); larger, unbounded steps would compound into massive rate swings within a single RTT, e.g., a 1.05 gain would double the rate in just 75 ms. Appendix E provides a stability analysis of this bounded multiplicative update.

The gain function $\gamma(\eta)$ is defined using two shifted sigmoid functions relative to the target $\eta^* = 0.2$:

$$\gamma(\eta) = \begin{cases} 0.02 \cdot \sigma(\beta \cdot \eta) + 0.98, & 0 < \eta \leq \eta^* \\ 0.01 \cdot \sigma(\beta \cdot (\eta - 0.4)) + 1.00, & \eta > \eta^* \end{cases}$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the standard sigmoid function. Based on our empirical tuning, we use a steepness factor $\beta = 35$. This configuration provides a responsive gradient, allowing the controller to ramp up faster during bandwidth availability, favoring throughput while maintaining stability.

Loop Transition. Loop transition is controlled by both the overuse state and the GBR ratio. When overuse is not reported, GBR-CC uses the GBR-driven inner loop. Once overuse is reported and the GBR ratio stays above the target η^* for 100 ms, GBR-CC transitions to the outer loop, which uses GCC’s delay-based multiplicative decrease to reduce the sending rate. This prevents transient delay spikes from triggering the outer loop when the cellular scheduler is still satisfying the sender’s demand. As the queue drains, the one-way delay gradient falls below GCC’s adaptive threshold and the overuse detector stops reporting overuse. GBR-CC then transitions back to the inner loop. Here, overuse is GCC’s delay-gradient signal, while $\eta > \eta^*$ indicates cellular scheduler saturation.

Handover Handling. During handover, GBR-CC may temporarily lose fresh GBR samples because the BSR/Grant updates used to compute GBR can be interrupted. In our experiments, the transmission interruption caused by handover is short, typically around 50–70 ms. GBR-CC treats such missing or stale GBR samples as invalid. If no fresh GBR sample is available and overuse is not reported, GBR-CC freezes the GBR-driven update and pauses transmission; if overuse is reported, the outer loop controls the rate. Once fresh GBR samples resume after handover, GBR-CC updates GBR normally and uses the inner loop unless overuse is active.

4.1.3 Implementation

We implement GBR-CC in Microsoft’s msquic [36]. A Cellular-RatioReceiver runs on a dedicated thread to parse BSR and grant messages from CellNinja [32] via IPC, preventing blocking I/O on the QUIC worker threads. We use lock-free `std::atomic` variables to share the latest GBR (η) with the congestion controller. During `OnPacketAck` events, the controller polls this value; if a fresh sample (< 10 ms) is available, it applies the adaptation law (Eq. 2). Otherwise, it freezes the GBR-driven update and pauses transmission. This zero-copy design ensures millisecond-scale RAN dynamics drive pacing with negligible overhead.

4.2 GBR-Driven Real-Time Video Delivering

We adapt GBR-CC for WebRTC to enhance real-time video conferencing performance over cellular uplinks. While a direct port of the GBR-CC functions correctly, it is suboptimal for video applications because the objectives of generic congestion control differ from those of real-time media delivery. A standard transport controller aims to maximize throughput and minimize network latency. In contrast, WebRTC aims to maximize QoE, which is defined by high video quality, minimal freeze rates, and, crucially, consistently low end-to-end playout latency.

The Jitter Buffer Trade-off. Playout latency differs significantly from network RTT; it includes encoding, decoding, and the adaptive

jitter buffer delay. The jitter buffer is essential for absorbing network delay variation to ensure smooth playback. Crucially, WebRTC’s jitter buffer logic accounts for frame size variations to prevent stalling on large frames.

We observe a critical trade-off when applying the highly responsive GBR-CC to video. If the GBR-CC tracks millisecond-scale RAN capacity variations too closely, it causes the target bitrate to fluctuate rapidly. The video encoder responds to these rate changes by frequently adjusting video quality, resulting in significant variance in the size of encoded frames. WebRTC receiver interprets this high frame size variance as a stability risk and proactively expands the jitter buffer depth to prevent underflow. Consequently, while the network RTT may remain low, the playout latency increases due to the inflated buffer, degrading the interactivity of the call.

QoE-Aware Parameter Tuning. To address this, we optimize the rate adaptation law for video traffic. The goal is to prioritize rate stability over perfect capacity tracking when the network is near the target operating point. We modify the sigmoid gain function $\gamma(\eta)$ defined in Eq. 2 by adjusting the steepness parameter β . Specifically, for the general transport case, we utilized $\beta = 35$ to ensure rapid convergence. For the WebRTC implementation, we increase the steepness to $\beta = 50$. As illustrated in Figure 10b, this adjustment effectively widens the *neutral zone* (the plateau where $\gamma(\eta) \approx 1$) around the target ratio η^* . By sharpening the transition edges, we expand the range of GBR values for which the controller maintains a stable sending rate. This desensitizes the controller to minor RAN fluctuations while retaining the ability to react aggressively to significant congestion or bandwidth availability.

This tuning reduces the sending rate variance, allowing the receiver’s jitter buffer to drain and maintain a smaller size. Although this approach may theoretically sacrifice a fraction of the available throughput during transient capacity spikes, it significantly improves the overall QoE by reducing end-to-end playout latency and frame freezes, as demonstrated in our evaluation.

4.2.1 Implementation

We integrated GBR-CC into WebRTC’s GoogCC module. The `CellularRatioReceiver` captures telemetry on a background thread and posts GBR updates as high-priority tasks to WebRTC’s `ProcessThread`, ensuring thread safety. We modified `AimdRateControl` to prioritize these signals: when valid GBR data is present, the controller bypasses standard AIMD logic for our sigmoid-based adaptation (Eq. 2). This directly drives the `PacedSender` and encoder bitrate, allowing immediate reaction to radio resource scarcity and preventing jitter buffer inflation.

5 Evaluation

We evaluate GBR-CC through extensive real-world experiments over commercial LTE, 5G FDD, and 5G TDD networks to demonstrate its ability to adapt to rapid cellular dynamics.

Purely Wild Experiment. We conduct all evaluations on commercial cellular networks rather than network emulators or USRP-based testbed. GBR-CC fundamentally requires a live commercial cellular network for two reasons. First, the GBR signal is generated by the specific interaction between the UE and the base station; without a real cellular stack, this signal is unobtainable. Second, even small-scale testbeds built using USRPs cannot truthfully represent the

complex, heavy background traffic and multi-user contention of a commercial network that drive rapid network dynamics.

Testbed and Network Configuration. Our experimental testbed consists of a sender (Dell Laptop) and a receiver (Google Cloud Platform server). The sender connects to the cellular network via USB tethering to a smartphone, ensuring that all traffic traverses the commercial uplink path. We utilize three distinct smartphones to cover different radio access technologies: a Pixel 5 for LTE, a OnePlus 9 for 5G FDD, and a Moto G 5G for 5G TDD. For mobile tests, the device moves outdoors at a speed of approximately 1.5 m/s along a repeatable walking route.

5.1 GBR-CC Transport Performance

In this section, we evaluate the fundamental transport performance of GBR-CC using continuous bulk data transfers.

Baselines and Metrics. We compare GBR-CC against seven state-of-the-art congestion control algorithms representing diverse design philosophies. These include the loss-based CUBIC [22] (standard TCP/QUIC default), the model-based BBR [13], delay-based algorithms such as Copa [8], Verus [54], and Sprout [44], and learning-based approaches including PCC Vivace [17] and PCC [16]. We utilize the Pantheon framework to run these baseline algorithms. We compare the algorithms based on Throughput and RTT.

Experimental Protocol We utilize `secnetperf` to perform bulk data uploads under both static and mobility scenarios. To ensure fairness and mitigate the impact of background traffic, we employ a round-robin interleaved testing schedule. For each network scenario (LTE, 5G FDD, 5G TDD), we collect five groups of measurements. Inside each group, the execution order of the algorithms is rotated. Each individual run transfers a fixed payload of 200 MB for LTE experiments and 1 GB for 5G experiments. These sizes are selected to ensure the session lasts long enough to capture steady-state queuing dynamics and valid statistical distributions.

5.1.1 Experimental Results

Figure 11 plots the throughput versus RTT across stationary and mobility scenarios, demonstrating that GBR-CC consistently achieves the superior operating point.

In static environments, GBR-CC effectively navigates the conflict between utilization and delay where baselines fail. In LTE (Figure 11a), it reduces median RTT by nearly 4× compared to loss-based schemes like CUBIC and Verus (dropping from >400 ms to 104 ms) while delivering 2.5× the throughput of delay-sensitive algorithms like Sprout and Copa. This advantage persists in 5G TDD (Figure 11c), where GBR-CC sustains a median throughput of 25.97 Mbps, 53% higher than Sprout and Copa, while concurrently maintaining a median RTT of 71 ms, which is 2× lower than BBR and CUBIC.

Under mobility, GBR-CC’s advantage widens as it tracks rapid channel fluctuations that destabilize traditional controllers. In 5G FDD mobility (Figure 11e), GBR-CC achieves the highest median throughput (11.98 Mbps) while maintaining a median RTT of 111 ms, 4.5× lower than CUBIC, which suffers from severe bufferbloat (reaching 501 ms). Similarly, in 5G TDD mobility (Figure 11f), GBR-CC improves throughput by 1.7× compared to delay-based schemes while keeping latency 3× lower than loss-based baselines. These results confirm that GBR-CC’s millisecond-scale coupling with

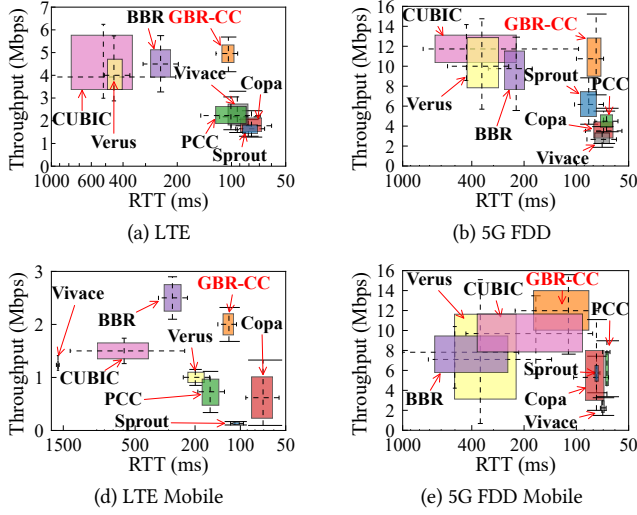


Fig. 11 – Throughput and RTT of GBR-CC and seven other CC, across LTE, 5G FDD, and 5G TDD uplinks under both stationary and mobility scenarios.

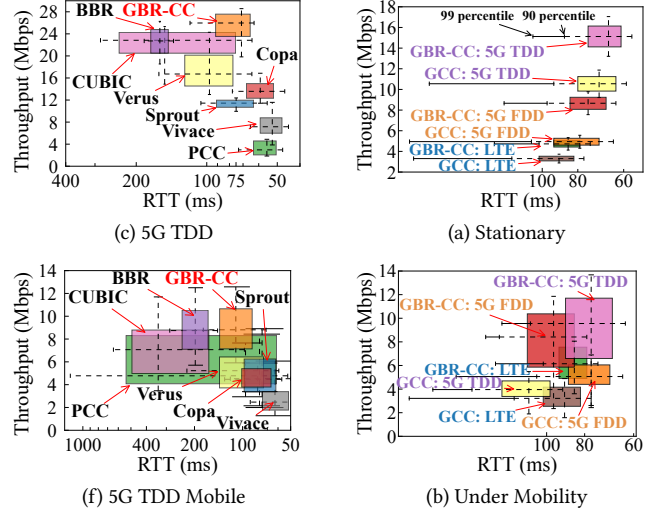


Fig. 12 – Network performance of GBR-driven WebRTC.

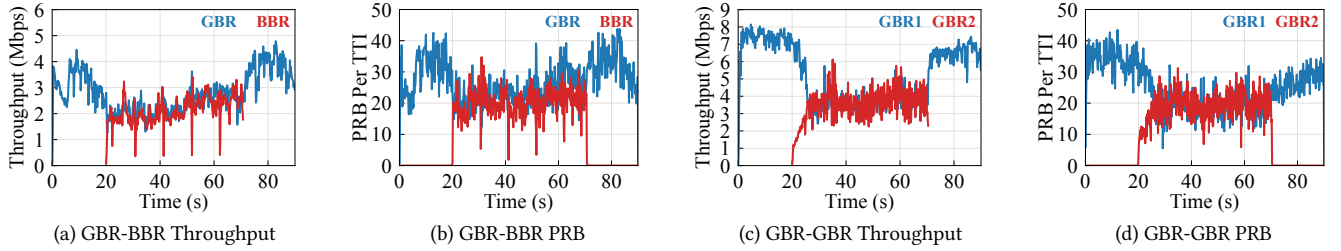


Fig. 13 – Coexistence fairness under simultaneous cellular uplink flows. Throughput panels report end-to-end delivery rate, and PRB panels show the average PRBs per TTI granted to each controlled UE, decoded from the cellular control channel.

the scheduler maximizes throughput while preventing the queue buildup that plagues end-to-end controllers.

5.2 Fairness

In this section, we first explain that GBR-CC inherits fairness from the base station scheduler, and then use same-cell multi-UE experiments to correlate end-to-end throughput with per-UE PRB allocations observed by NG-Scope [47].

Fairness of Base Station Scheduling. In our proposed approach, the inter-UE fairness on the cellular uplink is fundamentally guaranteed by the base station’s MAC scheduling algorithm (e.g., Proportional Fair). The base station’s scheduler is inherently designed to actively manage resource allocation; it structurally prevents any single UE from being starved, nor does it allow a highly aggressive UE to monopolize the shared wireless spectrum. Instead, it comprehensively evaluates the real-time traffic demand of all active UEs to distribute resources fairly.

Fairness Under a Cellular Bottleneck. GBR-CC natively leverages this scheduler-enforced fairness. By design, a GBR-CC sender continuously probes for bandwidth by increasing its sending rate until the GBR value indicates that the base station is no longer fully satisfying its requested rate (i.e., the scheduler is capping its grants

due to cell load). Once this threshold is reached, GBR-CC reduces and stabilizes its sending rate to match exactly what the base station is willing to grant. Crucially, even if another competing UE employs a highly aggressive congestion control algorithm (such as CUBIC or BBR), the base station’s scheduler will not permit that aggressive flow to seize the entire bandwidth. The scheduler will inherently throttle the aggressive UE’s grants, forcing the aggressive flow and the GBR-CC flow to reach a stable, fair coexistence based on the base station’s allocation policy.

Fairness Under a Non-Cellular Bottleneck. If the network bottleneck shifts outside the cellular link (e.g., to a shared wired backhaul), GBR-CC seamlessly falls back to its outer control loop, which relies on GCC’s delay-based rate adaptation logic rather than forcing GBR-based control. In such scenarios, the fairness on the wired network is governed by GCC’s standard mechanisms.

Ground-Truth Validation. Figure 13 validates fairness using controlled, same-cell two-UE experiments. We test GBR-CC vs. BBR and GBR-CC vs. GBR-CC, aligning throughput with per-UE PRB allocations decoded by NG-Scope [47] from the cellular control channel. The PRB panels show only the two controlled UEs in each experiment. After the competing flow starts, throughput converges

to a shared operating point and both UEs continue receiving PRB grants, showing that neither flow monopolizes the uplink.

5.3 GBR-Driven WebRTC Evaluation

In this section, we demonstrate the benefits of GBR-Driven WebRTC for interactive real-time video applications. We use a high-fidelity 120-second YUV sequence from the Microsoft Video Call Dataset (VCD) as the source.

Baselines and Metrics. We compare GBR-Driven WebRTC directly against the legacy WebRTC. We evaluate performance across two distinct categories of metrics. First, we analyze network performance by comparing the RTT and throughput achieved by GBR-CC and GCC to understand the underlying transport behavior. Second, we assess the user-perceived end-to-end QoE. This includes End-to-End Playout Latency, defined as the total photon-to-photon delay comprising encoding, queuing, transmission, buffering, and decoding time. We also measure the Freeze Rate, calculated as the ratio of total freeze duration to the session duration, and Visual Quality using full-reference metrics such as VMAF (perceptual quality) and LPIPS (perceptual distance).

5.3.1 Network Performance: Throughput and RTT

We compare GBR-CC against GCC under LTE and 5G uplinks, with a specific focus on the 99th percentile RTT (tail latency). For real-time video, tail latency is critical: unlike median RTT, delay spikes cause frames to miss playout deadlines, directly resulting in freezes.

Figure 12 reports the throughput and RTT distributions. Across all static scenarios, GBR-CC demonstrates a decisive advantage in suppressing these latency spikes. In LTE and 5G FDD, GBR-CC cuts the 99th percentile RTT by approximately 40–45% compared to GCC. In 5G TDD, the improvement is even more dramatic, with the tail latency being effectively halved. Crucially, this latency reduction does not come at the cost of capacity; GBR-CC simultaneously boosts median throughput by $1.5\times$ to nearly $2\times$ across these networks. This indicates that GBR-CC effectively drains standing queues while fully utilizing the available bandwidth. Under mobility, GBR-CC maintains stability despite rapid channel fluctuations. Across LTE and 5G TDD, it doubles throughput while reducing the 99th percentile RTT by over 30%.

5.3.2 User-Perceived End-to-End QoE

Figure 14 compares the QoE between GBR-Driven WebRTC and legacy WebRTC across both stationary and mobility settings. The results demonstrate that GBR-CC transforms the interactive experience by simultaneously reducing delays and preventing stalls under rapidly varying cellular uplinks.

Playout Latency. The most significant gain is in interactivity. In stationary scenarios, GBR-CC drastically cuts the median end-to-end playout latency—reducing it by more than half (53%) in 5G FDD and by 32% in 5G TDD. This reduction is critical for video conferencing, as it pushes the delay closer to the real-time interaction threshold and makes responses feel more immediate.

Freeze Rate. Crucially, this latency reduction does not compromise stability; in fact, fluidity improves dramatically. GBR-CC reduces the median video freeze rate by approximately 60% across LTE and 5G TDD networks. By synchronizing with the scheduler, the controller prevents the buffer underflows that cause playback to stall, even when uplink capacity fluctuates.

Visual Quality. GBR-CC also eliminates the severe quality dips that plague legacy algorithms. While average VMAF scores improve by 19–37%, the most substantial improvement is in the tail of the distribution—the worst-case quality users actually notice. The 10th-percentile VMAF scores improve by roughly $2\times$ compared to GCC, ensuring that even during deep congestion, the video remains clear rather than degrading into heavy pixelation.

Performance Under Mobility. These benefits persist in dynamic environments. Under mobility, GBR-CC maintains its robustness, reducing freeze rates by over 40% across all networks and lowering latency by 35% in 5G TDD. This confirms that the GBR signal allows the controller to react fast enough to handle handovers and rapid channel fading without breaking the video stream.

5.3.3 Ablation Study: QoE-Aware Tuning

To validate our QoE-aware parameter tuning, we performed an ablation study comparing the aggressive adaptation parameter ($\beta = 35$) selected for bulk transport against the desensitized parameter ($\beta = 50$) designed for real-time video. We conducted head-to-head WebRTC experiments over commercial LTE networks to isolate the impact of rate stability on video performance. The results, presented in Figure 15, reveal a critical disconnect between raw network metrics and user-perceived quality.

Network Performance vs. Playout Latency. Figure 15(a), plots the throughput and latency performance. We observe a distinct divergence between transport and application layers. For pure GBR-CC (generic transport), the aggressive $\beta = 35$ achieves higher throughput by tightly tracking channel capacity. However, for GBR-Driven WebRTC, Figure 15(a) shows that $\beta = 50$ actually yields slightly higher throughput than $\beta = 35$. More importantly, $\beta = 50$ achieves this capacity with significantly lower end-to-end latency.

Mechanism of Improvement. This latency gain stems from the interaction between the controller and the jitter buffer. The aggressive rate fluctuations driven by $\beta = 35$ force the video encoder to vary frame sizes rapidly. In response, the receiver’s adaptive jitter buffer inflates to handle the variance, increasing end-to-end delay. By using $\beta = 50$, we stabilize the target bitrate; this allows the jitter buffer to drain, significantly improving interactivity. Figure 15(c) confirms that this latency reduction does not compromise reliability. The freeze rates remain comparable between the two settings, indicating that the wider neutral zone of $\beta = 50$ successfully maintains the video stream without underflowing the buffer.

6 Related Work

End-to-End Congestion Control. Traditional algorithms [8, 10, 11, 13, 16, 22, 23, 43, 44, 50, 54] infer congestion from end-to-end signals like packet loss or delay. In cellular networks, these signals suffer from two fundamental problems. First, they are lagged: RAN schedulers operate at sub-millisecond frequencies, while end-to-end feedback is delayed for RTT, forcing protocols to react to stale states. Second, they are inaccurate: lacking visibility into RAN resource availability, they rely on noisy symptoms (loss or delay) rather than the root cause. In contrast, GBR-CC’s inner loop synchronizes directly with the base station’s millisecond-scale scheduling to address both issues.

Network-Assisted and Explicit Feedback. To address wireless opacity, proposals like L4S [12, 42], ABC [21], XRC [26], Octopus [14],

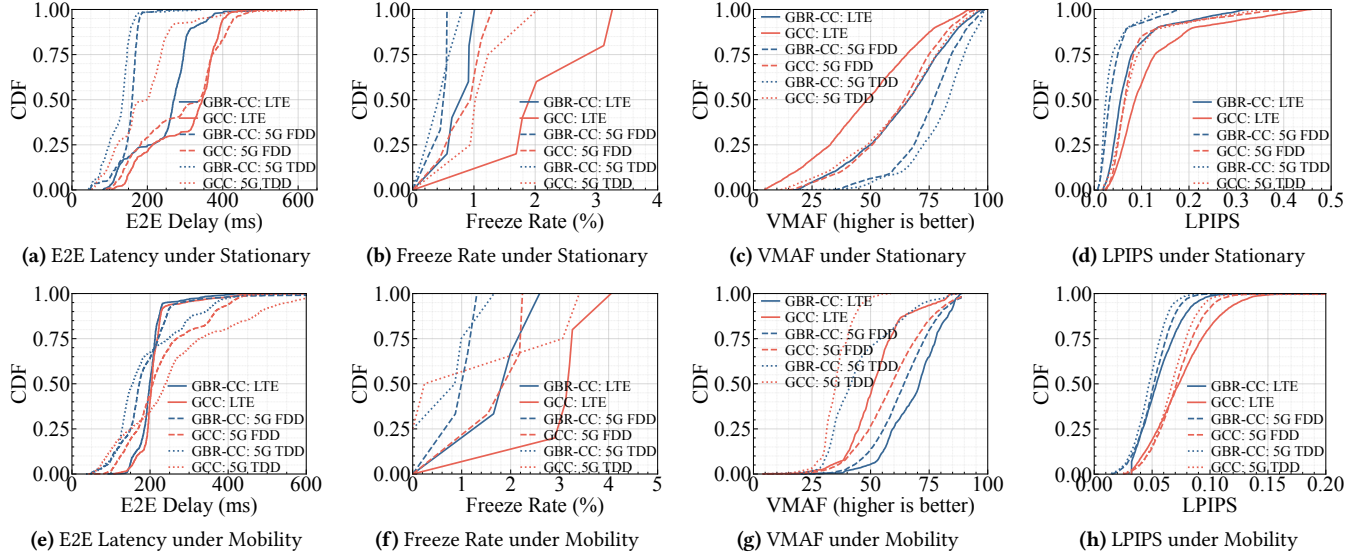


Fig. 14 – QoE comparison between GBR-Driven WebRTC and legacy WebRTC, across LTE, 5G FDD, and 5G TDD uplinks under both stationary and mobility scenarios: (a,e) E2E playout latency; (b,f) Freeze rate; (c,g) VMAF; and (d,h) LPIPS.

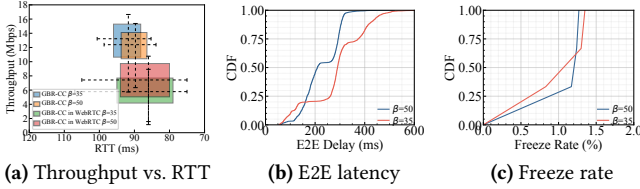


Fig. 15 – Impact of steepness parameter β on: (a) network performance, (b) end-to-end playout latency, and (c) freeze rate.

Sidekick [52], and RAPID [15] use infrastructure signals or proxies to expose capacity. While accurate, these methods face prohibitive deployment barriers, requiring modifications to proprietary base stations or standards. GBR-CC achieves comparable visibility by exploiting the standard BSR-Grant procedure, requiring no infrastructure modifications.

Client-Side Cross-Layer Inference. Existing client-side tools infer link quality but face practicality issues. Tools like NG-Scope [47], NR-Scope [41], and piStream [45] utilize physical layer sniffing to provide precise bandwidth estimation; however, they require expensive, bulky Software-Defined Radios (SDRs), making them impractical for ubiquitous use. Conversely, UE-side telemetry approaches such as CLAW [46], PERCEIVE [29], H2MO [33], Prism5G [51], CQIC [34], Biscay [27], X-TCP [9], and LinkForecast [53] avoid infrastructure changes, but they mainly infer radio conditions or bandwidth changes from local indicators such as RSRQ, resource-allocation patterns, handover events, or carrier aggregation. These local metrics do not directly capture aggregate cell contention, and prior extraction pipelines are often too delayed for real-time active congestion control. GBR-CC overcomes these limitations by using the Grant-to-BSR Ratio as a real-time proxy for global cell load. This signal is accessible on commodity devices without external hardware, directly exposes scheduler-side allocation constraints, and makes RAN contention actionable for congestion control.

7 Discussion

Fundamentally, GBR-CC is designed as a general-purpose congestion control algorithm, not a video-specific mechanism. The WebRTC-specific extension was introduced subsequently because interactive video is uniquely sensitive to bitrate oscillation and jitter. The core GBR-driven controller, however, is highly effective for other general uplink applications.

8 Conclusion

We identify the GBR as a fast and accurate cell-load indicator that infers the base station’s uplink load directly from a commodity mobile device in real time, requiring no infrastructure support. We use GBR to enable GBR-CC, a dual-loop congestion controller for cellular uplinks. We further integrate GBR-CC into WebRTC with QoE-aware tuning, allowing the same RAN signal to guide real-time video without destabilizing the encoder and jitter buffer. Compared to BBR, GBR-CC achieves 5% higher average throughput while reducing the 50th-percentile RTT by 2 $\times$; compared to GCC, GBR-CC achieves 50% higher average throughput while simultaneously reducing median end-to-end playout latency by 32–53% and freeze rate by approximately 60%. These results show that exposing scheduler-level congestion signals on commodity devices is a practical way to bridge modem-layer visibility with adaptation at both the transport and application layers.

Acknowledgments

We would like to thank our shepherd and the anonymous reviewers for their thoughtful insights and constructive feedback, which helped us improve our work. This material is based upon work supported by the National Science Foundation under Grant Nos. CNS-2223556, AST-2232457, and OAC-2429485. Jamieson gratefully acknowledges a gift from Nokia Corporation in the Princeton University NextG Corporate Affiliates Program.

References

- [1] 3GPP. 2018. *Medium Access Control (MAC) protocol specification*. Technical Report (TR) 38.321. 3rd Generation Partnership Project (3GPP). https://www.etsi.org/deliver/etsi_ts/138300_138399/138321/15.03.00_60/ts_138321v150300p.pdf
- [2] 3GPP. 2025. *5G; NR; Radio Resource Control (RRC) protocol specification*. Technical Report (TR) 38.331. 3rd Generation Partnership Project (3GPP). https://www.etsi.org/deliver/etsi_ts/138300_138399/138331/19.00.00_60/ts_138331v190000p.pdf
- [3] 3GPP. 2025. *LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Medium Access Control (MAC) protocol specification*. Technical Report (TR) 36.321. 3rd Generation Partnership Project (3GPP). https://www.etsi.org/deliver/etsi_ts/136300_136399/136321/19.00.00_60/ts_136321v190000p.pdf
- [4] 3GPP. 2025. *LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Radio Resource Control (RRC) protocol specification*. Technical Report (TR) 36.331. 3rd Generation Partnership Project (3GPP). https://www.etsi.org/deliver/etsi_ts/136300_136399/136331/19.00.00_60/ts_136331v190000p.pdf
- [5] 3GPP. 2025. *NR; Physical layer procedures for data*. Technical Specification 3GPP TS 38.214. 3rd Generation Partnership Project (3GPP). https://www.etsi.org/deliver/etsi_ts/138200_138299/138214/19.01.00_60/ts_138214v190100p.pdf Release 19.
- [6] Accuver. 2025. XCAL Mobile Network Drive Test Software. <https://www.accuver.com/products/network-optimization/XCAL>
- [7] Adnan Aijaz and A. Hamid Aghvami. 2015. Realizing the Tactile Internet: Haptic Communications over Next Generation 5G Cellular Networks. (2015). <https://arxiv.org/pdf/1510.02826> arXiv preprint.
- [8] Venkat Arun and Hari Balakrishnan. 2018. Copa: Practical Delay-Based Congestion Control for the Internet. In *Proceedings of the Applied Networking Research Workshop* (Montreal QC Canada, 2018-07-16). ACM, 19–19. doi:10.1145/3232755.3232783
- [9] Tommy Azzino, Matteo Drago, Michele Polese, Andrea Zanella, and Michele Zorzi. 2017. X-TCP: A Cross Layer Approach for TCP Uplink Flows in mmWave Networks. arXiv:1706.00904 [cs.NI] doi:10.48550/arXiv.1706.00904
- [10] Niklas Blum, Serge Lachapelle, and Harald Alvestrand. 2021. WebRTC: real-time communication for the open web platform. 64, 8 (2021), 50–54. doi:10.1145/3453182
- [11] Lawrence S. Brakmo, Sean W. O'Malley, and Larry L. Peterson. 1994. TCP Vegas: new techniques for congestion detection and avoidance. 24, 4 (1994), 24–35. doi:10.1145/190809.190317
- [12] Bob Briscoe, Koen De Schepper, Marcelo Bagnulo, and Greg White. 2023. Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture. RFC 9330. doi:10.17487/RFC9330
- [13] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. 2016. BBR: Congestion-Based Congestion Control: Measuring bottleneck bandwidth and round-trip propagation time. 14, 5 (2016), 20–53. doi:10.1145/3012426.3022184
- [14] Yongzhou Chen, Ammar Tahir, Francis Y. Yan, and Radhika Mittal. 2023. Octopus: In-Network Content Adaptation to Control Congestion on 5G Links. In *IEEE/ACM Symposium on Edge Computing (SEC)* (2023-12), 199–214. doi:10.1145/3583740.3628438 ISSN: 2837-4827.
- [15] Mamoutou Diarra, Walid Dabbous, Amine Ismail, and Thierry Turletti. 2021. Ran-aware proxy-based flow control for high throughput and low delay embb. In *Proceedings of the 24th International ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems*. 41–50.
- [16] Mo Dong, Qingxi Li, Doron Zarchy, Godfrey P. Brighten, and Michael Schapira. 2015. PCC: Re-architecting Congestion Control for Consistent High Performance. (2015).
- [17] Mo Dong, Tong Meng, Doron Zarchy, Engin Arslan, Yossi Gilad, Brighten Godfrey, and Michael Schapira. 2018. PCC Vivace: Online-Learning Congestion Control. (2018).
- [18] Rostand A. K. Fezeu, Jason Carpenter, Rushikesh Zende, Sree Ganesh Lalitaditya Divakarla, Nitin Varyani, Faaqi Bilal, Steven Sleder, Nanditha Naik, Duncan Joly, Eman Ramadan, Ajay Kumar Gurumadaiah, and Zhi-Li Zhang. 2025. Teleoperating Autonomous Vehicles over Commercial 5G Networks: Are We There Yet? arXiv:2507.20438 [cs.NI] <https://arxiv.org/abs/2507.20438>
- [19] Moinak Ghoshal, Imran Khan, Z. Jonny Kong, Phuc Dinh, Jiayi Meng, Y. Charlie Hu, and Dimitrios Koutsonikolas. 2023. Performance of Cellular Networks on the Wheels. In *Proceedings of the 2023 ACM Internet Measurement Conference (IMC '23)*. doi:10.1145/3618257.3624814
- [20] Google. 2025. Google Beam: Our AI-first 3D video communication platform. <https://blog.google/innovation-and-ai/technology/research/project-starline-google-beam-update/>. Accessed: 2026-02-07.
- [21] Prateesh Goyal, Mohammad Alizadeh, and Hari Balakrishnan. 2020. ABC: A Simple Explicit Congestion Controller for Wireless Networks. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)* (2020).
- [22] Sangtae Ha, Injong Rhee, and Lisong Xu. 2008. CUBIC: a new TCP-friendly high-speed TCP variant. 42, 5 (2008), 64–74. doi:10.1145/1400097.1400105
- [23] Habtegebrel Haile, Karl-Johan Grinnemo, Simone Ferlin, Per Hurtig, and Anna Brunstrom. 2023. Copa-D: Delay Consistent Copa for Dynamic Cellular Networks. In *2023 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*. 508–513. doi:10.1109/EuCNC/6GSummit58263.2023.10188233
- [24] Junxian Huang, Feng Qian, Yihua Guo, Yuanyuan Zhou, Qiang Xu, Z. Morley Mao, Subhabrata Sen, and Oliver Spatscheck. 2013. An In-depth Study of LTE: Effect of Network Protocol and Application Behavior on Performance. In *Proceedings of the ACM SIGCOMM 2013 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*. 363–374. doi:10.1145/2486001.2486006
- [25] Rostand A. K. Fezeu, Claudio Fiandrino, Eman Ramadan, Jason Carpenter, Lilian Coelho De Freitas, Faaqi Bilal, Wei Ye, Joerg Widmer, Feng Qian, and Zhi-Li Zhang. 2024. Unveiling the 5G Mid-Band Landscape: From Network Deployment to Performance and Application QoE. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney NSW Australia, 2024-08-04). ACM, 358–372. doi:10.1145/3651890.3672269
- [26] Morteza Kheirkhah, Mohamed M Kassem, Gorrry Fairhurst, and Mahesh K Marina. 2022. XRC: An explicit rate control for future cellular networks. In *ICC 2022-IEEE International Conference on Communications*. IEEE, 2290–2296.
- [27] Jon Larrea, Tanya Shreedhar, and Mahesh K. Marina. 2025. BISCA: Practical Radio KPI Driven Congestion Control for Mobile Networks. arXiv:2509.02806 [cs.NI] doi:10.48550/arXiv.2509.02806
- [28] Jason Lawrence, Dan B. Goldman, Supreeth Achar, Gregory Major Blascovich, Joseph G. Desloge, Tommy Fortes, Eric M. Gomez, Sascha Häberling, Hugues Hoppe, Andy Huibers, Claude Knaus, Brian Kuschak, Ricardo Martin-Brualla, Harris Nover, Andrew Ian Russell, Steven M. Seitz, and Kevin Tong. 2021. Project Starline: A High-Fidelity Telepresence System. *ACM Transactions on Graphics* 40, 6, Article 242 (Dec. 2021), 16 pages. doi:10.1145/3478513.3480490
- [29] Jinsung Lee, Sungyong Lee, Jongyun Lee, Sandesh Dhawaskar Sathyanarayana, Hyoyoung Lim, Jihoon Lee, Xiaoqing Zhu, Sangeeta Ramakrishnan, Dirk Grunwald, Kyunghan Lee, and Sangtae Ha. 2020. PERCEIVE: deep learning-based cellular uplink prediction using real-time scheduling patterns. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services* (Toronto, Ontario, Canada) (*MobiSys '20*). Association for Computing Machinery, New York, NY, USA, 377–390. doi:10.1145/3386901.3388911
- [30] Yuanjie Li, Chunyi Peng, Zengwen Yuan, Jiayao Li, Haotian Deng, and Tao Wang. 2016. Mobileinsight: extracting and analyzing cellular network information on smartphones. In *Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking* (New York City New York, 2016-10-03). ACM, 202–215. doi:10.1145/2973750.2973751
- [31] Yuanjie Li, Chunyi Peng, Zengwen Yuan, Jiayao Li, Haotian Deng, and Tao Wang. 2016. Mobileinsight: extracting and analyzing cellular network information on smartphones. In *Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking* (New York, NY, USA, 2016) (*MobiCom*). Association for Computing Machinery, 202–215. doi:10.1145/2973750.2973751
- [32] Yuxin Liu, Tianyang Zhang, Kyle Jamieson, and Yaxiong Xie. 2025. Seeing Through the Fog: Empowering Mobile Devices to Expose and Mitigate RAN Buffer Effects on Delay-Sensitive Protocols. arXiv:2507.00337 [cs.NI] <https://arxiv.org/abs/2507.00337>
- [33] Zhutian Liu, Qing Deng, Zhaowei Tan, Zhiyun Qian, Xinyu Zhang, Ananthram Swami, and Srikanth V. Krishnamurthy. 2024. M2HO: Mitigating the Adverse Effects of 5G Handovers on TCP. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking* (Washington D.C., DC, USA) (*ACM MobiCom '24*). Association for Computing Machinery, New York, NY, USA, 1089–1103. doi:10.1145/3636534.3690680
- [34] Feng Lu, Hao Du, Ankur Jain, Geoffrey M. Voelker, Alex C. Snoeren, and Andreas Terzis. 2015. CQIC: Revisiting Cross-Layer Congestion Control for Cellular Networks. In *Proceedings of the 16th International Workshop on Mobile Computing Systems and Applications (HotMobile '15)*. ACM. doi:10.1145/2699343.2699345
- [35] Zili Meng, Yaning Guo, Chen Sun, Bo Wang, Justine Sherry, Hongqiang Harry Liu, and Mingwei Xu. 2022. Achieving consistent low latency for wireless real-time communications with the shortest control loop. In *Proceedings of the ACM SIGCOMM 2022 Conference* (Amsterdam, Netherlands) (*SIGCOMM '22*). Association for Computing Machinery, New York, NY, USA, 193–206. doi:10.1145/3544216.3544225
- [36] Microsoft. 2025. Microsoft/msquic. <https://github.com/microsoft/msquic>
- [37] Arvind Narayanan, Xumiao Zhang, Ruiyang Zhu, Ahmad Hassan, Shuowei Jin, Xiao Zhu, Xiaoxuan Zhang, Denis Rybkin, Zhengxuan Yang, Zhuoqing Morley Mao, Feng Qian, and Zhi-Li Zhang. 2021. A variegated look at 5G in the wild: performance, power, and QoE implications. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (*SIGCOMM '21*). Association for Computing Machinery, New York, NY, USA, 610–625. doi:10.1145/3452296.3472923
- [38] P1 Security. 2025. P1sec/QCSuper. <https://github.com/P1sec/QCSuper>
- [39] TikTok [n. d.]. About TikTok. <https://www.tiktok.com/about>. Accessed: 2026-02-07.

- [40] Twitch Interactive, Inc. [n. d.]. About Twitch. <https://www.twitch.tv/p/about/>. Accessed: 2026-02-07.
- [41] Haoran Wan, Xuyang Cao, Alexander Marder, and Kyle Jamieson. 2024. NR-Scope: A Practical 5G Standalone Telemetry Tool. In *Proceedings of the 20th International Conference on emerging Networking EXperiments and Technologies* (Los Angeles CA USA, 2024-12-09). ACM, 73–80. doi:10.1145/3680121.3697808
- [42] Haoran Wan and Kyle Jamieson. 2025. L4Span: Spanning Congestion Signaling over NextG Networks for Interactive Applications. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–25.
- [43] Keith Winstein and Hari Balakrishnan. [n. d.]. TCP ex Machina: Computer-Generated Congestion Control. ([n. d.]).
- [44] Keith Winstein, Anirudh Sivaraman, and Hari Balakrishnan. 2013. Stochastic forecasts achieve high throughput and low delay over cellular networks. In *Proceedings of the 10th USENIX Conference on Networked Systems Design and Implementation* (Lombard, IL) (*nsdi'13*). USENIX Association, USA, 459–472.
- [45] Xiufeng Xie, Xinyu Zhang, Swarun Kumar, and Li Erran Li. 2015. piStream: Physical Layer Informed Adaptive Video Streaming over LTE. In *Proceedings of the 21st Annual International Conference on Mobile Computing and Networking* (Paris, France) (*MobiCom '15*). Association for Computing Machinery, New York, NY, USA, 413–425. doi:10.1145/2789168.2790118
- [46] Xiufeng Xie, Xinyu Zhang, and Shilin Zhu. 2017. Accelerating Mobile Web Loading Using Cellular Link Information. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services* (Niagara Falls, New York, USA) (*MobiSys '17*). Association for Computing Machinery, New York, NY, USA, 427–439. doi:10.1145/3081333.3081367
- [47] Yaxiong Xie and Kyle Jamieson. 2022. NG-Scope: Fine-Grained Telemetry for NextG Cellular Networks. 6, 1 (2022), 1–26. doi:10.1145/3508032
- [48] Yaxiong Xie, Fan Yi, and Kyle Jamieson. 2020. PBE-CC: Congestion Control via Endpoint-Centric, Physical-Layer Bandwidth Measurements. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)* (Virtual Event USA, 2020-07-30). ACM, 451–464. doi:10.1145/3387514.3405880
- [49] Dongzhu Xu, Anfu Zhou, Xinyu Zhang, Guixian Wang, Xi Liu, Congkai An, Yiming Shi, Liang Liu, and Huadong Ma. 2020. Understanding Operational 5G: A First Measurement Study on Its Coverage, Performance and Energy Consumption. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (*SIGCOMM '20*). Association for Computing Machinery, New York, NY, USA, 479–494. doi:10.1145/3387514.3405882
- [50] Francis Y. Yan, Jestin Ma, Greg D. Hill, Deepti Raghavan, Riad S. Wahby, Philip Levis, and Keith Winstein. 2018. Pantheon: the training ground for internet congestion-control research. In *Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) (*USENIX ATC '18*). USENIX Association, USA, 731–743.
- [51] Wei Ye, Xinyue Hu, Steven Sleder, Anlan Zhang, Udhaya Kumar Dayalan, Ahmad Hassan, Rostand A. K. Fezeu, Akshay Jajoo, Myungjin Lee, Eman Ramadan, Feng Qian, and Zhi-Li Zhang. 2024. Dissecting Carrier Aggregation in 5G Networks: Measurement, QoE Implications and Prediction. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (*ACM SIGCOMM '24*). Association for Computing Machinery, New York, NY, USA, 340–357. doi:10.1145/3651890.3672250
- [52] Gina Yuan, Matthew Sotoudeh, David K. Zhang, Michael Welzl, David Mazières, and Keith Winstein. 2024. Sidekick: {In-Network} Assistance for Secure {End-to-End} Transport Protocols. 1813–1830. <https://www.usenix.org/conference/nsdi24/presentation/yuan>
- [53] Chaoqun Yue, Ruofan Jin, Kyoungwon Suh, Yanyuan Qin, Bing Wang, and Wei Wei. 2018. LinkForecast: Cellular Link Bandwidth Prediction in LTE Networks. *IEEE Transactions on Mobile Computing* 17, 7 (2018), 1582–1594. doi:10.1109/TMC.2017.2756937
- [54] Yasir Zaki, Thomas Pötsch, Jay Chen, Lakshminarayanan Subramanian, and Carmelita Görg. 2015. Adaptive Congestion Control for Unpredictable Cellular Networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication* (London, United Kingdom) (*SIGCOMM '15*). Association for Computing Machinery, New York, NY, USA, 509–522. doi:10.1145/2785956.2787498
- [55] Yuming Zhang, Yan Liu, Lingfeng Guo, and Jack YB Lee. 2022. Measurement of a Large-Scale Short-Video Service Over Mobile and Wireless Networks. (2022).

APPENDIX

Appendices are supporting material that has not been peer-reviewed.

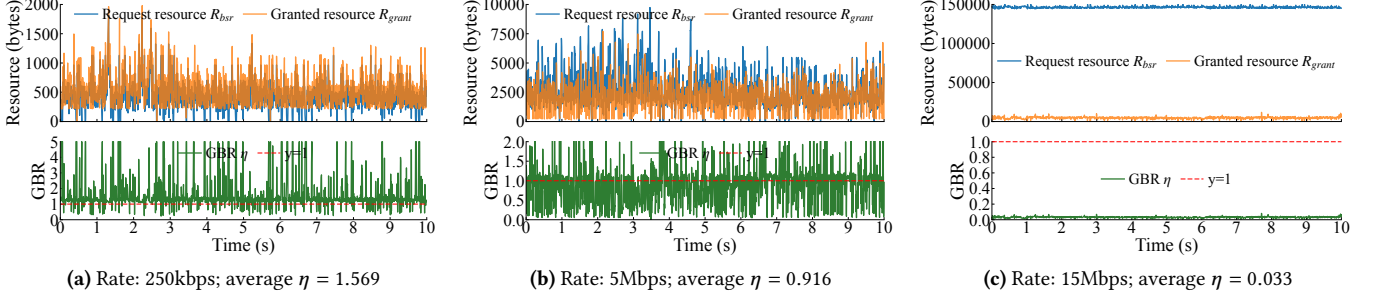


Fig. 16 – Temporal dynamics of requested resource R_{bsr} and granted resources R_{grant} across three load regimes. (a) Uncongested: grants exceed demand via scheduler padding ($\eta > 1$). (b) Matched: grants tightly track demand at the link capacity ($\eta \approx 1$). (c) Saturated: grants are clipped below requested demand due to cell congestion ($\eta \ll 1$).

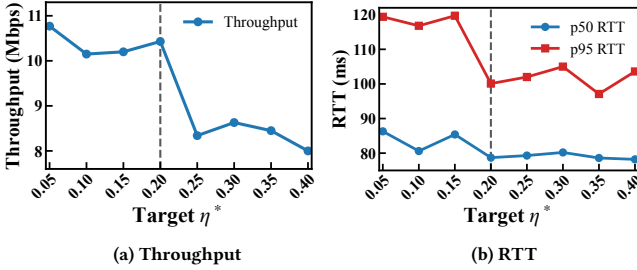


Fig. 17 – Sensitivity of GBR-CC to the target η^* .

A Visualizing GBR on MediaTek Devices

To illustrate the temporal dynamics of the GBR, Figure 16 plots the real-time requested demand (R_{bsr}) from a MediaTek UE and granted resources (R_{grant}) from the cell, across three representative 10-second traces with different uplink sending rates.

Uncongested Regime (Figure 16a). When the UE transmits at a low rate of 250 Kbps, the base station frequently over-allocates resources. Here, R_{grant} consistently exceeds R_{bsr} , resulting in an average GBR of 1.569.

Near-Capacity Regime (Figure 16b). At a moderate rate of 5 Mbps, the scheduler tightly tracks the UE's demand. The granted resources closely match the reported buffer status, keeping the ratio near one (average GBR = 0.916).

Saturated Regime (Figure 16c). When the sending rate exceeds available capacity (15 Mbps), the scheduler is forced to clip the UE's requests to accommodate physical layer limits or other users. R_{grant} falls significantly below R_{bsr} over sustained intervals, and the GBR collapses to nearly zero (average 0.033), providing an unmistakable, real-time signal of RAN-internal congestion.

B Sensitivity and Adaptive Tuning of the Target η^*

We evaluate the sensitivity of GBR-CC to η^* by sweeping η^* from 0.05 to 0.40. Figure 17 shows that GBR-CC is not highly sensitive to small changes in η^* : the default $\eta^* = 0.2$ achieves high throughput (10.43 Mbps), low median RTT (78.7 ms), and low p95 RTT (100.1 ms). Smaller targets can keep throughput high but increase tail

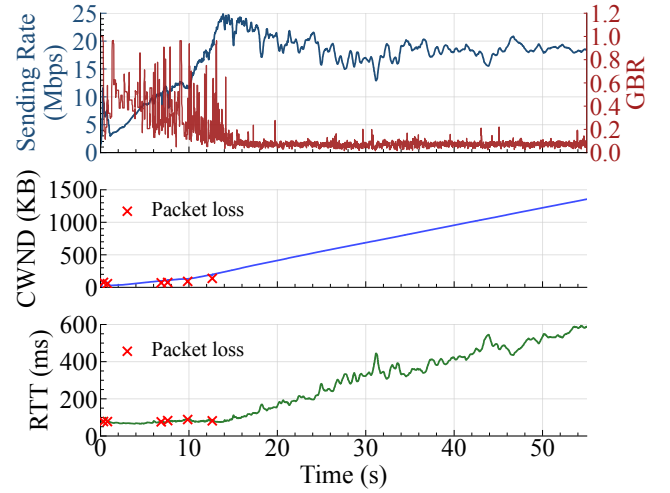


Fig. 18 – CUBIC on a cellular uplink. Top: sending rate (blue) and GBR (red). Middle: congestion window with packet loss events (red markers). Bottom: RTT over time. CUBIC grows its window until deep RAN buffers overflow and loss occurs, causing persistently low η and high queuing delay.

RTT, while larger targets reduce throughput without a clear latency benefit. GBR-CC currently uses a fixed target $\eta^* = 0.2$, selected empirically from our measurement campaigns. This design does not yet adapt the target to operator-specific or scheduler-specific behavior. A natural extension is to use the GBR signal itself to profile scheduler behavior online and adjust η^* accordingly. We leave adaptive η^* tuning to future work.

C CUBIC Under Cellular Uplink Dynamics

Figure 18 shows the core failure mode of CUBIC on cellular uplinks. CUBIC is loss-based: it keeps increasing its congestion window until packet loss occurs and only then reduces its rate. In the trace, however, η stays close to zero for a long period, showing that the base station's uplink grants are far below the UE's requested demand. CUBIC cannot see this scheduler-side scarcity and therefore continues to grow its window even though the cellular uplink can

no longer satisfy the sender’s demand. The excess packets build up in the RAN queue, and the end-to-end RTT keeps increasing until the queue eventually overflows and packet loss appears. Thus, CUBIC reacts after the damage has already occurred: loss arrives only after severe queue accumulation and sustained delay inflation.

D Complete GBR Parsing Workflow

Table 1 summarizes the modem messages and fields used to compute GBR. To use these logs, GBR-CC first obtains the *PeriodicBSR-Timer* from the RRC Reconfiguration message. It then parses each BSR report to obtain the UE’s reported buffer demand, and parses uplink grant messages to obtain the granted uplink resource. GBR-CC matches BSR reports and uplink grants using their radio-frame timing, sums the granted uplink bytes between two consecutive BSR reports, and computes GBR as the ratio between granted resource and reported buffer demand.

Information	Log source	Parsed fields
PeriodicBSR-Timer	RRC Reconfiguration	BSR-Config PeriodicBSR-Timer
BSR value	LTE: MAC UL Transport Block 5G: NR5G L2 UL TB	BSR control element LCG ID buffer demand radio-frame timing
Uplink grant	LTE: ML1 DCI Information Report 5G: NR5G MAC DCI Info 5G: NR5G MAC UL Physical Channel Schedule Report	MCS redundancy version TBS index or scheduled PUSCH TBS resource-block assignment

Table 1: Modem messages and fields used to compute GBR.

E Stability of the GBR-Driven Gain Mapping

The GBR-CC control loop runs at 200 Hz and uses a bounded multiplicative per-sample update. Although the gain is capped at $\pm 1\%$, this bound is only the maximum adjustment applied when the measured GBR ratio is far from the target. The controller does not repeatedly apply the maximum gain near the operating point. Instead, as η approaches η^* , the sigmoid gain γ approaches 1, so each update becomes a small correction and the sending rate stabilizes around the target.

This behavior is also observed in our real experiments. During the first 1–2 seconds, the pacing rate can grow quickly, roughly by 2–3 $\times$ per second, allowing the sender to find the available cellular uplink bandwidth. After this short startup phase, η moves close to η^* , γ remains near 1, and the sending rate stabilizes around the target rather than producing large oscillations.

F RAN Congestion in Real-World Traces

In our real-world uplink experiments, LTE uplink throughput is typically below 20 Mbps and 5G uplink throughput is typically below 50 Mbps, much lower than the wired path capacity. Therefore, for uplink traffic in our setting, the bottleneck is usually in the cellular link. In GBR-CC, $\eta < \eta^*$ indicates RAN congestion because the base station grants less uplink resource than the UE requests.

We observe this pattern frequently in our traces. In the BBR trace, η stays close to zero for most of the run, indicating that RAN

congestion dominates much of the execution. In the GCC trace, RAN congestion is more bursty and mainly concentrates during multiplicative-increase phases.

G GBR-Aware Variants of Existing CCAs

GBR is not tied to GCC specifically. Its role is to provide fast RAN-side visibility that can complement an end-to-end congestion-control loop. In this paper, we use GCC as the outer-loop basis because our primary target is real-time uplink communication, where delay, jitter, freeze rate, and playout latency are critical. GCC is the default congestion-control algorithm in WebRTC and is already designed for delay-sensitive real-time traffic, making it a natural foundation for GBR-Driven WebRTC. Incorporating GBR into other congestion-control algorithms, such as BBR or CUBIC, is a promising direction for future work.

H Security and Privacy

Compared to SDR-based approaches that physically sniff and decode the control channels of all users in a cell, GBR-CC presents a strictly lower security and privacy risk. GBR-CC relies exclusively on local telemetry: it only reads the modem information of the local User Equipment (UE) and does not possess the capability to expose the control traffic or payloads of other users. We acknowledge, however, that raw diagnostic (DIAG) traces can contain sensitive RRC/NAS messages, device identifiers, and cell/operator configurations. Therefore, raw DIAG interfaces should not be exposed directly to user-space applications.

To mitigate this, our design aligns with the principle of least privilege: the application-layer congestion controller (GBR-CC) does not require raw log access. It only requires two highly aggregated, per-UE metrics: the Buffer Status Report (BSR) and the corresponding Uplink Grants. These isolated scheduling signals merely represent the local device’s demand and the base station’s allowance; they do not reveal any user payload, peer traffic patterns, or sensitive network configurations. In a production deployment, the OS network stack or baseband firmware could abstract this data, exposing only the computed Grant-to-BSR Ratio via a secure, unprivileged socket API, analogous to how TCP_INFO exposes RTT, entirely bypassing the need for diagnostic mode or root access.